香山昆明湖 V2

# 自定义向量点积指令

## 第一阶段：环境部署与向量加法验证

设计、验证与性能评估

三阶段工程技术报告

2026 年 11 月 8 日修订

# 目录

**第一阶段：环境部署与向量执行过程** **1**

**1.1 协同验证框架与实验环境** **1**

**1.2 部署与构建过程** **2**

1.2.1 资源配置与工具位置 . . . . . 2

1.2.2 固定源码与工具安装 . . . . . 3

1.2.3 集成构建与操作记录 . . . . . 4

**1.3 共同启动输入与差分故障定位** **5**

**1.4 hello 输出与三类 vadd 结果** **5**

1.4.1 输入配置与结果解释 . . . . . 5

1.4.2 10 月 9 日环境验证 . . . . . 6

1.4.3 10 月 25 日双模型复核 . . . . . 7

**1.5 向量执行与波形观察** **7**

1.5.1 10 月 9 日事件与原始截图 . . . . . 7

1.5.2 10 月 25 日含时钟的调试记录 . . . . . 8

**1.6 版本与复现索引** **10**

1.6.1 源码与工具版本 . . . . . 10

1.6.2 10 月 25 日复核的产物摘要 . . . . . 11

1.6.3 文件与记录索引 . . . . . 11

**1.7 操作日志摘录** **12**

1.7.1 10 月 9 日工作负载构建 . . . . . 12

1.7.2 10 月 9 日正常运行摘要 . . . . . 12

1.7.3 10 月 25 日生产模型输出 . . . . . 13

1.7.4 10 月 25 日 FST 模型输出 . . . . . 13

# 第一阶段：环境部署与向量执行过程

本阶段对应赛题的环境部署验证、操作日志和向量加法执行过程，主要结果有三项：一是在 WSL2 中搭建了香山处理器仿真、NEMU 参考模型和测试程序的完整构建与协同仿真环境；二是定位并修复了两端启动代码不一致导致的差分失败；三是三类 vadd 用例共 24 个元素在 DiffTest 下全部通过，并用波形观察了向量加法从执行到提交的过程。

实验分两次进行：10 月 9 日完成环境搭建和首次验证，10 月 25 日复核了英文 hello 输出、两种仿真模型的差分运行和含时钟的波形。第1.6、1.7节集中列出版本信息和日志摘录，供复核使用。自定义 vdot 点积的设计与测试见第二阶段。

## 1.1 协同验证框架与实验环境

香山 EMU、NEMU 参考模型和裸机程序共同构成验证环境。EMU 模拟硬件执行，NEMU 按指令语义更新参考状态，DiffTest 在指令提交时比较两者的架构状态。裸机程序提供可重复的输入和结果检查；共同启动 flash 负责在复位后跳转到程序入口。hello 检查程序启动与串口输出，vadd 则覆盖向量配置、加载、按元素加法、存储和提交。

硬件侧由 Java 与 Mill 解析 Scala/Chisel 工程，firtool 处理硬件中间表示，Verilator 与宿主 C++ 编译器生成 EMU。目标程序使用 RISC-V GCC 和 binutils 编译、链接并生成 ELF、BIN 及反汇编。NEMU 在宿主侧构建为参考库，由运行参数显式加载。三条构建链的产物在同一次差分运行中汇合，构建记录和运行记录通过产物的 SHA-256 摘要对应起来。表1-1列出各类产物的作用，以及复核时需要查看的主要记录。

表 1-1 构建产物与复核入口

| 产物 | 作用 | 主要记录 |
| --- | --- | --- |
| 香山 EMU | 执行 DUT 仿真 | 硬件源码、DUT 配置、生成命令、编译工具与文件 SHA-256 |
| NEMU 参考库 | 提供架构状态参考 | 固定源码、构建配置、实际加载路径与文件 SHA-256 |
| 程序 ELF/BIN | 执行 hello 和 vadd 用例 | ISA/ABI、入口、装载段、反汇编、镜像摘要 |
| 共同 flash | 建立一致的启动输入 | 启动汇编、12 字节镜像、反汇编与显式加载参数 |
| 运行记录 | 关联输入、结果及结束状态 | 原始日志、退出码、波形、事件 CSV 与信号索引 |

图1-1给出这一协同验证框架的整体结构。上半区为 WSL2 中的三条编译链：香山工程经 JDK、Mill、firtool 和 Verilator 生成 EMU，AM 程序与启动代码经 RISC-V GCC 生成共同的程序 BIN 和 boot-flash，NEMU 源码由宿主编译器构建为参考动态库。下半区为差分运行：EMU 与 NEMU 加载同一镜像，DUT 按架构提交推进参考模型，两端状态送入 DiffTest 比较，串口输出、波形、比较结果和退出状态统一归档。

10 月 9 日实验在 Windows 下的 WSL2 Ubuntu 22.04.5 运行，宿主处理器为 Ryzen 7 7840H，可

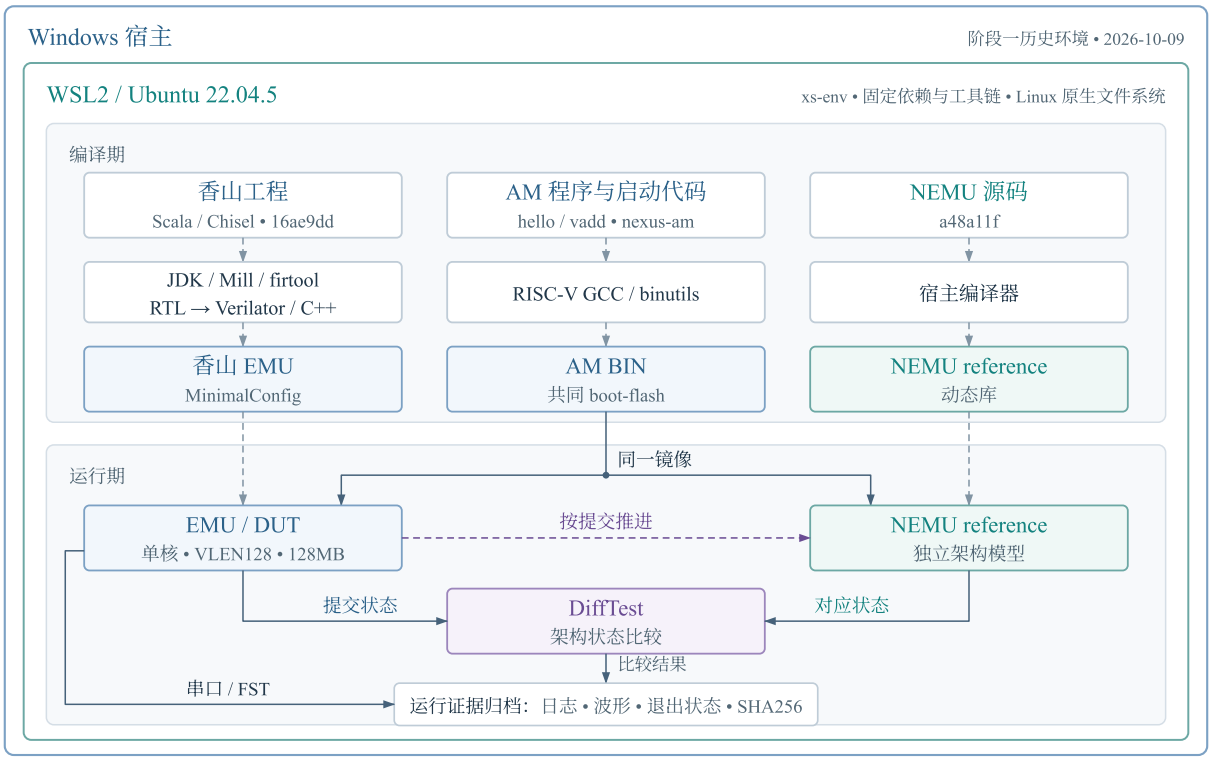



图 1-1　开发环境与软硬件协同验证框架

用物理内存约 27.7GiB。WSL 配置 20GB 内存和 16GB swap，构建与仿真放在 WSL 原生文件系统。DUT 为 MinimalConfig、单核、VLEN128，原波形构建采用 EMU_THREADS=1 和 EMU_TRACE=fst。运行时显式指定 128MB RAM、共同 flash 和固定参考库。

本阶段的源码版本、工具版本和 10 月 25 日复核的产物摘要集中列于本章末尾，两次实验所用的构建分开标注。

## 1.2　部署与构建过程

环境按本节步骤搭建。10 月 9 日依次完成了 NEMU 参考库、香山 EMU、测试程序和共同启动 flash 的构建，并完成首次差分运行，过程记录见表1-2。

### 1.2.1　资源配置与工具位置

WSL 资源设置位于 Windows 用户目录的%UserProfile%\.wslconfig：

```
[wsl2]
memory=20GB
swap=16GB
```

保存正在进行的 WSL 工作后，在 PowerShell 执行`wsl --shutdown`，重新进入 Ubuntu 并用`free -h`核对。原机器的交换文件位于 D:\WSL\wsl-swap.vhdx，其他机器按本机磁盘布局配置。内存规划同时覆盖 JVM、硬件生成工具、C++ 编译进程和文件缓存；频繁换页会增加构建时间，因此应根据可用物理内存调整 JVM 堆与编译并发。

Scala/Chisel、RISC-V 程序、EMU/NEMU 和波形处理都在 WSL 的 Linux 原生目录中执行。Windows 目录下的工作树则用 Windows 原生 Git 管理：从 WSL 跨文件系统访问 Windows 目录很

慢，实测一次 Git 检查约 300 秒仍未完成。源码与产物跨环境交换时，在接收端核对提交号和关键文件的 SHA-256，运行输出使用独立目录。

### 1.2.2 固定源码与工具安装

在团队仓库执行`bash team/scripts/prepare-dependencies.sh`，按版本锁初始化 xs-env、NEMU、nexus-am、DRAMsim3 及递归依赖。团队仓库的香山子模块使用`git clone --recurse-submodules` 恢复；NEMU 的 SimPoint 按子模块记录初始化，其余下载依赖按锁定脚本检出。已有目录的版本不一致时，先保留现场并核对输入；每次构建前记录所用的分支、提交号和未提交改动，便于事后复现。

基础工具的安装入口如下。这里的命令用于恢复固定环境，安装后再核对实际工具版本。

```
source team/scripts/activate.sh
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target base
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target llvm
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target mill
sudo apt-get install -y cmake ninja-build gtkwave
```

该入口来自固定 xs-env 的setup-tools.sh。软件源中的补丁版本会变化，环境检查负责记录安装后实际使用的版本。Mill 读取仓库的.mill-version 选择项目版本，Scala、Chisel 和 firtool 按项目解析结果核对。

xs-env 原有的 JDK 安装入口指向 Oracle 的 “21/latest” 地址，版本会随时间变化。团队改为固定安装 21.0.12 Linux x64 归档，并用官方 SHA-256 校验，安装到用户目录：

```
export JAVA_HOME="$HOME/.local/share/graalvm-jdk-21.0.12"
bash team/scripts/install-graalvm.sh
source team/scripts/activate.sh
```

新终端沿用相同的 JAVA_HOME；脚本不会覆盖已有的其他版本。参考资料为Oracle 安装说明、固定归档和官方校验值。

Verilator 从固定提交构建，显式选择已安装的 Clang，并按资源情况控制编译并发：

```
sudo apt-get install -y help2man perl autoconf flex bison
libgoogle-perftools-dev \
  libjemalloc-dev numactl perl-doc zlib1g-dev liblz4-dev libfl2 libfl-dev
mkdir -p "$HOME/riscv-tools"
git clone https://github.com/verilator/verilator.git
"$HOME/riscv-tools/verilator"
cd "$HOME/riscv-tools/verilator"
git checkout ea338be98e1e838d3518809ce8899f85a009963c
unset VERILATOR_ROOT
autoconf
./configure CC=/usr/local/llvm/bin/clang CXX=/usr/local/llvm/bin/clang++
LINK=/usr/local/llvm/bin/clang++
make -j2
sudo make install
```

安装后回到团队仓库根目录，运行 activate.sh 和 check-environment.sh。原环境目录为/home/riscv/riscv-lab/xs-env/XiangShan；activate.sh 将实际团队仓库设为 NOOP_HOME，依据 TEAM_XS_ENV_ROOT 选择 NEMU 和 AM，并设置 LINUX_GNU_TOOLCHAIN=1。向量工作负载构建显式使用`MARCH=rv64gcv`。

### 1.2.3 集成构建与操作记录

确认源码、工具和输出目录后，依次构建参考库、EMU、目标程序及共同 flash，再执行差分运行：

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
bash team/scripts/stage1.sh nemu
bash team/scripts/stage1.sh emu
TEAM_ID=< 团队或个人标识> bash team/scripts/stage1.sh workload
bash team/scripts/stage1.sh run
```

图1-2展示上述命令对应的执行流程。环境准备、按序构建和 DiffTest 运行依次进行，验证通过后确认结果并归档证据；检查、构建或运行未通过时，分别定位配置与依赖、构建日志或运行差分的原因，修正后返回对应环节重新执行。每一步无论成功或失败，都保留命令、日志、退出码和 SHA-256。

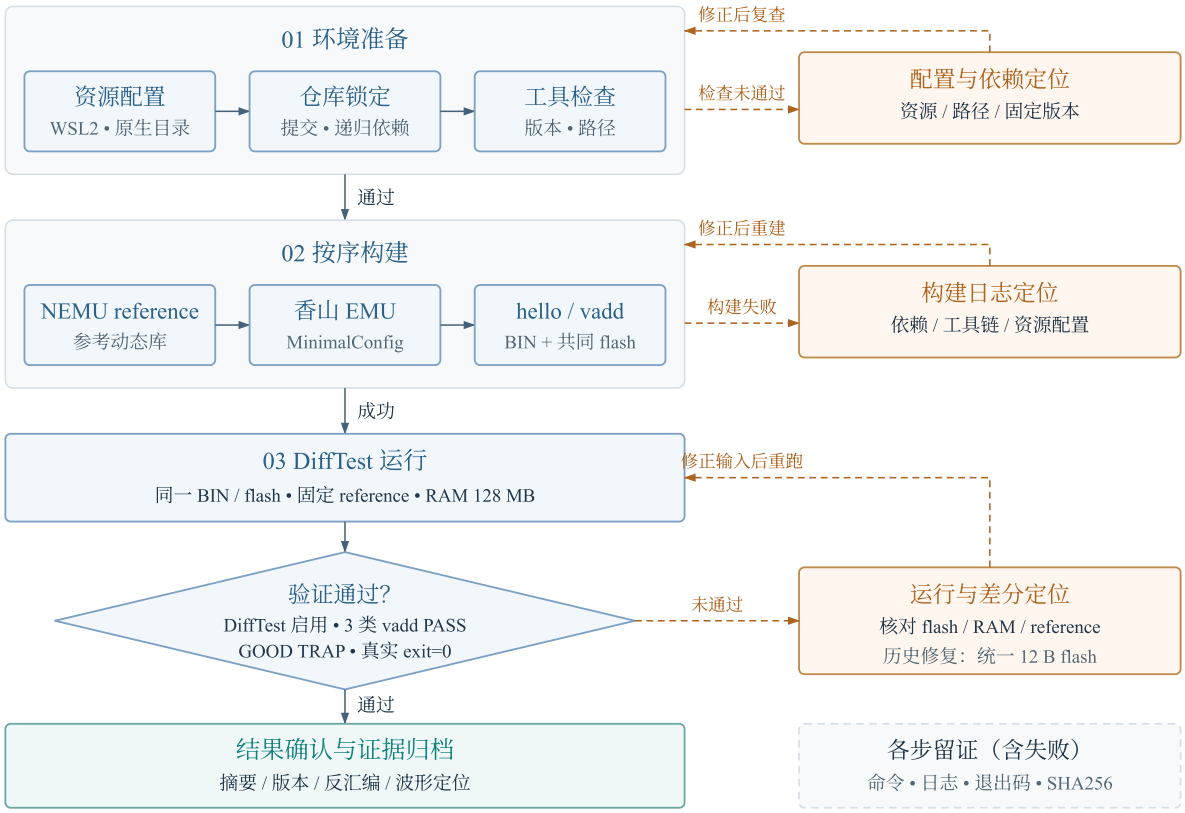



图 1-2 环境部署与验证执行流程

表1-2摘录 10 月 9 日构建后半段的主要操作及结果，时间为 UTC+8。20:42 时根据实际资源中断 C++ 编译，把并行度调整为 4 后继续，已完成的目标文件全部保留。

表 1-2 2026 年 10 月 9 日主要操作记录

| 时间 | 操作与结果 |
|---|---|
| 20:42:15 | 中断并恢复 C++ 构建，并行度调整为 4，保留已完成对象。 |
| 20:53:32 | EMU 构建完成，状态文件记录 exit_code=0。 |
| 20:55:26 | 工作负载与共同 flash 构建完成，记录 WORKLOAD_BUILD_PASS。 |

表 1-2 2026 年 10 月 9 日主要操作记录（续）

| 时间 | 操作与结果 |
|---|---|
| 21:00:28 | 正常差分运行结束，退出状态为 0。 |

这里的并行度只影响宿主机上的编译速度，与被测处理器的核数和 EMU 仿真线程数无关。

## 1.3 共同启动输入与差分故障定位

初次运行在 PC0x10000004 出现 t0 寄存器差异，记录的 instrCnt 为 2，尚处于启动代码。比较默认 flash 和反汇编后发现，两端分别使用了 3 条与 6 条启动指令。修复由同一份 boot-flash.S 生成 12 字节 boot-flash.bin，并通过运行参数显式提供给 DUT 与 NEMU。随后两端以共同启动输入进入同一工作负载，在 DiffTest 启用的条件下完成 hello/vadd 检查。

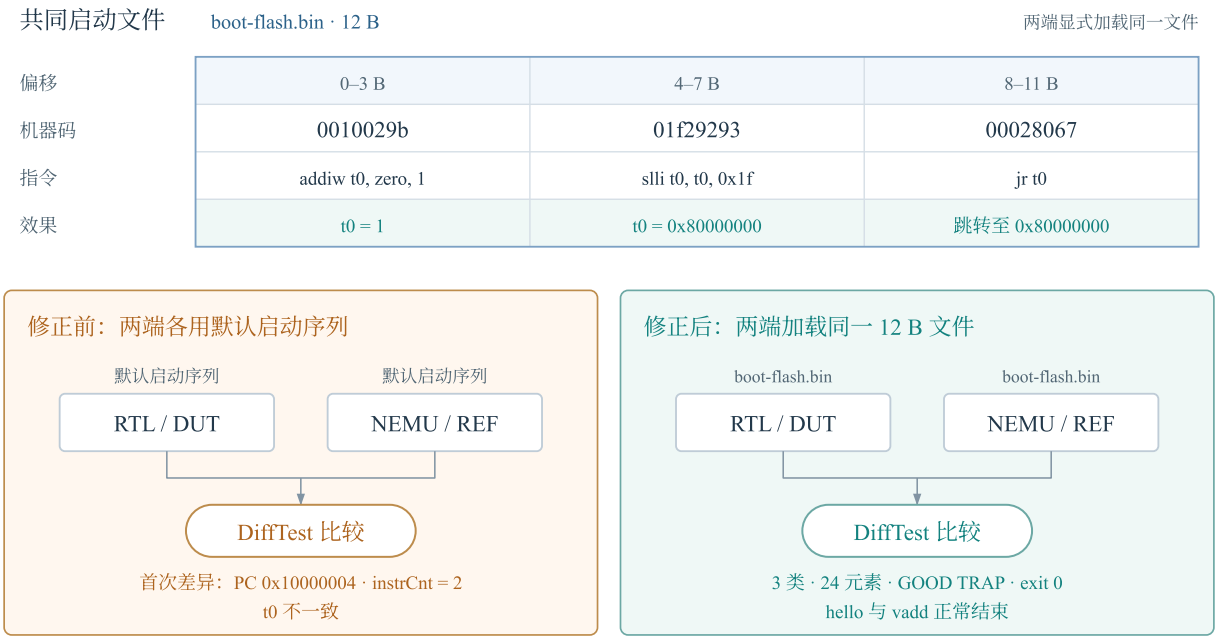



图 1-3 共同启动镜像与差分恢复

图1-3对照了修正前后的情况。上部表格逐字节列出共同启动文件：三条指令依次令 t0=1、将其左移为 0x80000000 并跳转至该地址，0–3、4–7 和 8–11 为文件内的字节偏移。下部左侧为修正前两端各用默认启动序列时的首次差异，右侧为两端显式加载同一文件后的结果：三类共 24 个元素通过比较，程序以 GOOD TRAP 和 exit 0 结束。

## 1.4 hello 输出与三类 vadd 结果

### 1.4.1 输入配置与结果解释

vadd 程序设置 SEW8、LMUL1、VL8，执行两次`vle8.v`、一次`vadd.vv`和一次向量存储，再逐字节比较标量参考。三个用例检查基本加法、负数补码和 8 位回绕，合计比较 24 个活动元素。

表1-3给出三个用例的输出，分别以日志中的逐元素字节和波形中的低 64 位十六进制两种形式表示。

表 1-3 vadd 输出的两种表示

| 用例 | 按元素顺序输出的字节 | 结果低 64 位 |
|---|---|---|
| 正数相加 | 9 9 9 9 9 9 9 9 | 0x0909090909090909 |
| 负数补码 | 254 252 250 248 246 244 242 240 | 0xf0f2f4f6f8fafcfe |
| 边界回绕 | 128 127 0 0 0 0 254 254 | 0xfefe000000007f80 |

每个元素按模 256 保留低 8 位。例如打印值 254 对应十六进制 FE，按 8 位补码解释为 $-2$。波形将多个字节合并显示为十六进制，右端为最低字节；日志按元素序号递增输出，因此两种书写顺序应按字节位置对应。VLEN128 给出寄存器宽度，VL8 给出本用例的活动元素数，表中结果覆盖低 64 位的八个元素。

### 1.4.2 10 月 9 日环境验证

10 月 9 日首次运行时团队标识尚未确定，hello 程序使用临时标识 riscv-vdot-team，因此输出中带有 `identity_status=placeholder`；显示的 IPv4 为 10.198.93.97，对应当时 WSL 宿主。10 月 25 日复核改用确定的标识重新构建，见下一小节。正常记录包含 HELLO_VADD_PASScases=3lanes=24、DiffTest 启用标记、GOOD TRAP 和退出状态 0；整程序统计为 16557 条指令、42962 周期，覆盖启动、UART 和结果校验。图1-4为当次运行结束时的终端输出，可以看到身份行、三类 vadd 的 PASS 结果、DiffTest 启用标记、GOOD TRAP 以及退出状态 0。

2026 年 10 月 9 日原始运行的终端输出

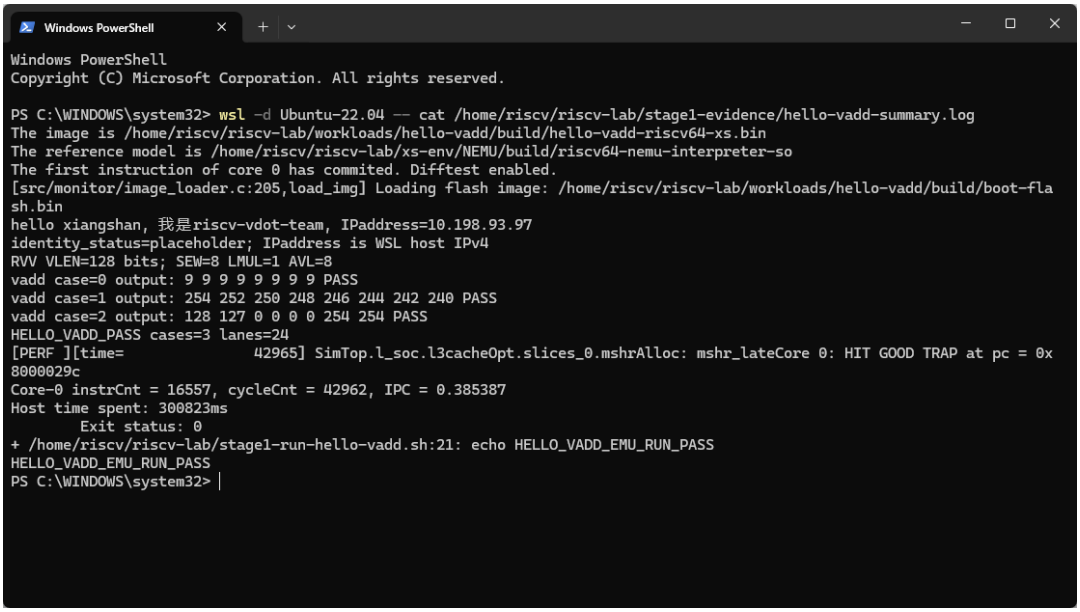



图 1-4 hello 与 vadd 正常结束输出

### 1.4.3 10 月 25 日双模型复核

10 月 25 日（UTC+8）的复核以成员标识 LiZufan001 重新构建英文 hello 程序，输出格式为赛题要求的 “hello xiangshan, I am xxx, IP address”，并标注地址来自构建宿主：

```
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
```

同一 BIN 分别在封存生产 EMU 和独立 FST 模型上运行，均加载普通 NEMU 参考库并启用 DiffTest。两次运行都通过三组、24 个元素检查，命中 GOOD TRAP 并正常退出；每次为 16621 条指令、43249 周期。这里的周期同样对应完整 hello/vadd 程序。表1-4列出两次运行的实际起止时间与结果，时间取自各自的运行回执。

表 1-4 10 月 25 日双模型运行记录（UTC+8）

| 模型 | 实际起止时间 | 运行结果 |
|---|---|---|
| 生产 EMU | 01:09:21–01:10:18 | DiffTest 通过，GOOD TRAP，exit 0 |
| 独立 FST 模型 | 01:28:57–01:31:24 | DiffTest 通过，GOOD TRAP，exit 0 |

FST 波形模型复用已有构建的 1807 个生成文件单独编译；用发布的源码重新编译得到的程序 BIN 和 flash，与实际运行时的输入逐字节相同。对应的验收记录、生产运行回执和FST 运行回执见团队仓库。

## 1.5 向量执行与波形观察

向量指令经取指、译码、重命名与调度进入执行单元。两个`vle8.v`提供输入，`vadd.vv`产生结果，随后经过写回与 ROB 提交。本文用提交 PC0x80000138、机器字 0x021101d7 及`vecwen=1`，`wdest=3`关联三类用例，并分别记录结果输出和架构提交两个观察点。

### 1.5.1 10 月 9 日事件与原始截图

表1-5沿用原 FST 提取 CSV 的事件坐标，列出三类用例在执行输入、结果输出和 ROB 提交三处的位置以及输出低 64 位。局部截图未显示时钟；截图中的 ps 只是波形工具的显示单位，数值为仿真记录的时间坐标，不代表实际时间。本表只用于定位事件，指令时延与性能在第三阶段用专门的测量窗口评估。

表 1-5 10 月 9 日 vadd 事件坐标

| 用例 | 输入 | 输出 | 提交 | 输出低 64 位 |
|---|---|---|---|---|
| 正数 | 19625 | 19626 | 19631 | 0x0909090909090909 |
| 负数 | 25460 | 25461 | 25466 | 0xf0f2f4f6f8fafcfe |
| 边界 | 33009 | 33010 | 33015 | 0xfefe000000007f80 |

图1-5把表1-5的坐标画成时间线。三类用例都在输入后 1 个周期给出结果，再经 5 个周期完成 ROB 提交；每个用例的三个事件都属于同一条 vadd 指令。

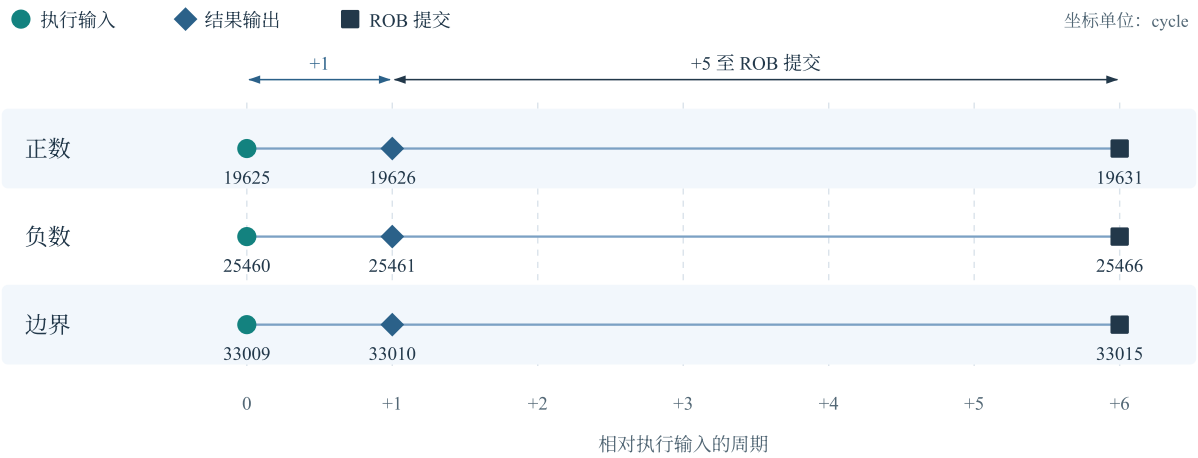



图 1-5 vadd 输入、输出与提交事件时间线

图1-6是正数用例的原始 GTKWave 截图：子图（a）的游标位于 19626 处的输出事件，三个事件的位置同时可见；子图（b）、（c）分别给出 19625 输入游标和 19631 提交游标处的信号值。图1-7为负数与边界用例的输出波形，两者的输出低 64 位分别为 F0F2F4F6F8FAFCFE 和 FEFE000000007F80，与表1-3一致。

原完整 FST 路径记录为/home/riscv/riscv-lab/stage1-evidence/hello-vadd.fst，大小 220295598 字节。复核可先从提交 PC 定位同次反汇编，再根据 CSV 打开波形窗口，最后联合查看时钟、valid/ready、操作数、结果及事务标识。事件 CSV 和波形提取记录见团队仓库的 team/evidence/stage1-20261009/。

### 1.5.2 10 月 25 日含时钟的调试记录

10 月 25 日的 FST 波形模型保留了实际时钟信号。从中选取两个向量 ALU（VIAluFix）实例和八个提交槽，共 127 个信号，逐个检查 33249 个时钟上升沿，得到九条事件：三次 FU 输入、三次 FU 输出和三次 DPI-C 提交观测。三笔 vadd 都在 exus_2 中的 VIAluFix 执行。

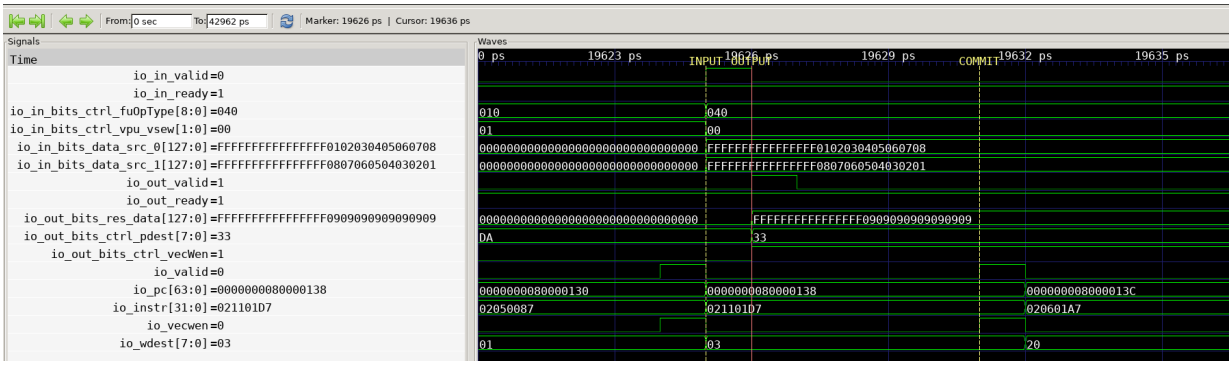
采样取每个上升沿之前、时钟低电平期间的稳定值，周期字段取当时的 DifftestTrapEvent.io_cycleCnt，FST 原始时戳另存。DPI-C 是仿真模型调用 C 函数的接口，提交记录经过 Rob.scala 中`delay=3` 的观察路径，表内 DPI-C 列因此表示提交接口的观测时刻。按上述规则得到的九条事件列于表1-6，表中同时给出每笔运算的 ROB flag、value 和目的物理寄存器。

表 1-6 10 月 25 日按时钟沿采样的事件记录

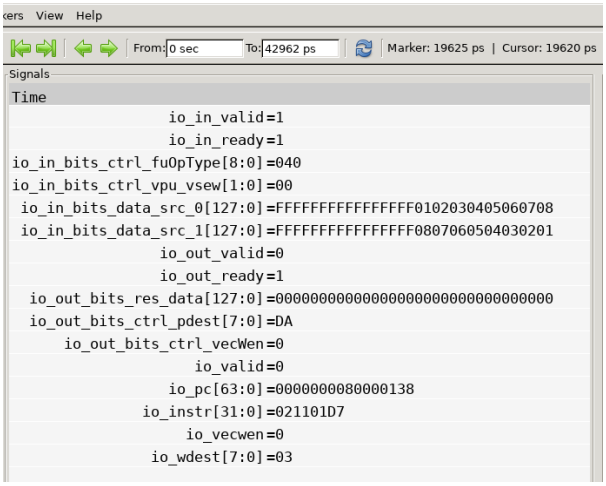
| 用例 | FU 输入 timer | FU 输出 timer | DPI-C 观测 timer | ROB flag/value/pdest |
|---|---|---|---|---|
| 正数 | 19525 | 19526 | 19531 | 1 / 13 / 51 |
| 负数 | 25526 | 25527 | 25532 | 0 / 21 / 57 |
| 边界 | 33193 | 33194 | 33199 | 1 / 25 / 63 |

三类数值结果与表1-3一致。表中的 timer 是周期计数。10 月 25 日的程序换用新标识重新编

（a）输出握手：游标 19626 ps，三个事件位置均可见




（b）输入游标 19625 ps




（c）提交游标 19631 ps

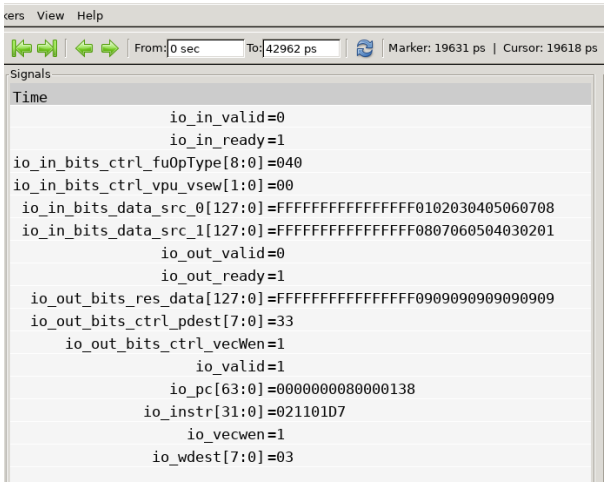
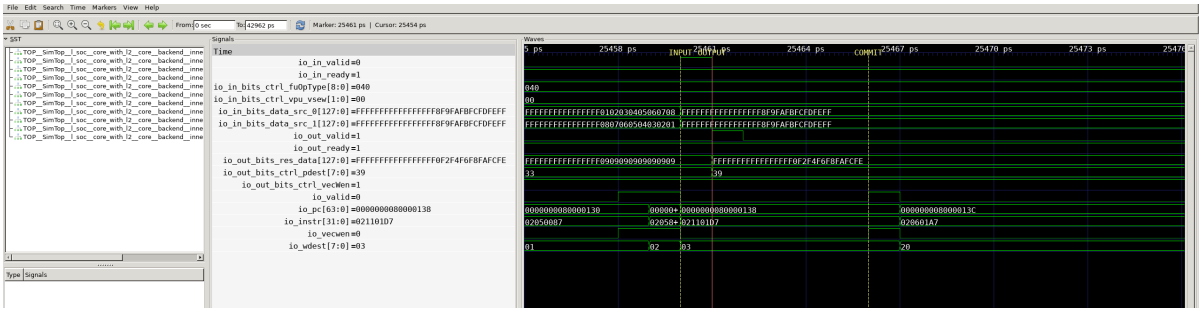



图 1-6　vadd 正数用例的执行与提交波形

（a）负数用例：输出游标 25461 ps



（b）边界用例：输出游标 33010 ps

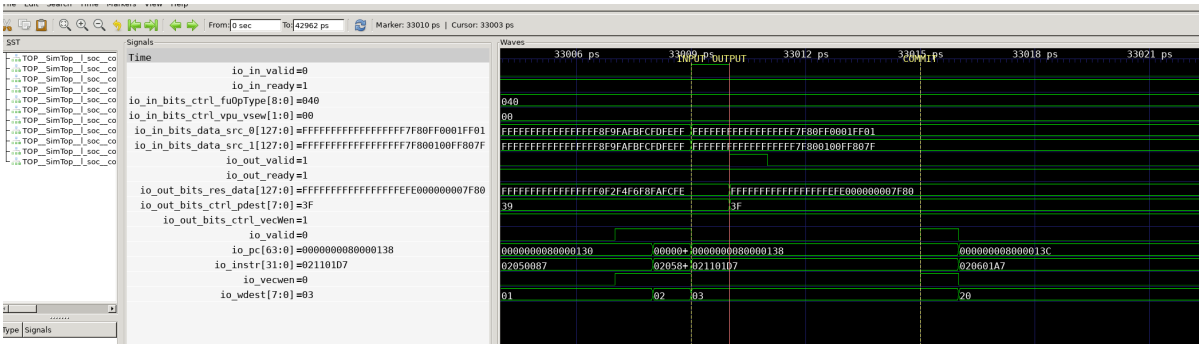


图 1-7　vadd 负数与边界用例结果波形

译，指令地址和执行时刻都与 10 月 9 日不同，因此表1-6与表1-5的绝对坐标不同；但每个用例从输入到输出相隔 1 个周期、从输出到提交相隔 5 个周期，两次完全一致。完整采样定义、信号清单、原始时戳及摘要保存在wave.json；只含所选信号的 FST 可直接用 GTKWave 查看。接口与观察路径分别对应固定源码的VIAluFix 包装层和Rob.scala。

提取工具本身也做了 24 项检查，覆盖日志完整性、退出状态、输入文件摘要和关键结果，并用缺少 FU 事件、缺半个周期、操作数未知和事务标识错误等反例确认工具能发现问题。

## 1.6 版本与复现索引

本节列出两次实验所用的源码、工具和产物，供复现和核对使用。

### 1.6.1 源码与工具版本

表1-7列出 10 月 9 日部署时固定的源码版本，完整递归依赖见团队仓库的 team/evidence/stage1-20261009/submodules-after-build.lock.txt。

表 1-7 10 月 9 日源码版本锁

| 组件 | 提交号 |
|---|---|
| xs-env | b0b506481e839a5df8a580e76f5aa246dc60fe7c |
| 香山基线 | 16ae9ddcda54fc9a2fddffad73174cf793ac7814 |
| NEMU | a48a11f208ff81ad765591d83271d1b339fb3861 |
| nexus-am | 086a238062fdfb3281b745a537b361983d0154b5 |
| DRAMsim3 | 1d2a9bf6da8e050d975ef2669c42faa93943f489 |
| DiffTest | 62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0 |
| yunsuan | cadd3c2f43096e253d61296b33ac697be8354e29 |

表 1-8 10 月 9 日工具版本

| 工具 | 版本 |
|---|---|
| Oracle GraalVM JDK / javac | 21.0.12 |
| LLVM / Clang | 19.1.7 |
| Mill | 项目版本 0.12.3 |
| Scala / Chisel | 2.13.15 / 6.6.0 |
| firtool | 1.62.1 |
| Verilator | 5.052，提交 ea338be98e1e838d3518809ce8899f85a009963c |
| RISC-V GCC / GNU binutils | 11.4.0（riscv64-linux-gnu-前缀）/ 2.38 |
| 宿主 GCC / G++ | 11.4.0 |
| Make / CMake / Ninja | 4.3 / 3.22.1 / 1.10.1 |
| Python / GTKWave | 3.10.12 / 3.3.104 |

表1-8为同一环境实际使用的工具版本，完整版本输出和 Ubuntu 软件包清单见同一目录的 tool-versions.txt 与 ubuntu-packages.lock.tsv。

### 1.6.2 10 月 25 日复核的产物摘要

10 月 25 日复核所用硬件构建的源码提交为 34bae6d4acceac939f958199dae53c14bc1f6183。表1-9列出这次运行主要产物的 SHA-256，与团队仓库 team/evidence/stage1-current-20261025/中的运行和验收记录一致。

表 1-9 10 月 25 日复核主要产物的 SHA-256

| 对象 | SHA-256 |
|---|---|
| 生产 EMU | ff840528eb1009e94906eefb3773ca321050f309553543ab26f0db23c410e30f |
| FST 模型 | 1ff3f457ff582d0c3155a15dafa1fa60b8275d9356e29a0b7806c9d3915ce9f7 |
| 普通 NEMU 参考库 | 8af118e5519db7ef6001010a2d547707c5d47630412e80abfc3f343bfef987c9 |
| 共同程序 BIN | d81437b44473cef2b0c5777330e13fd076567d4fc1a4296be35d971e51a4dcad |
| 共同 flash | a3360212cf7bb784119c1719c9258362f8567803601b2359aee2c104864fa994 |
| 完整 FST（181143183 字节） | cf8abe9104fb955c7dc4bf1efda6588a61060fce33857b9ddc6d5020421be299 |
| 只含所选信号的 FST（393288 字节） | 4c9f442704871010026654fd9d60854024b270a83be6354be5dee3ff25df24e1 |

### 1.6.3 文件与记录索引

运行源码位于团队仓库的 team/workloads/hello-vadd。10 月 9 日的证据在 team/evidence/stage1-20261009，10 月 25 日复核的证据在 team/evidence/stage1-current-20261025；第1.7节摘录的日志另存于报告工程的 evidence/目录。完整 FST 文件较大，按记录中的路径和 SHA-256 定位。各记录的用途见表1-10。

表 1-10 第一阶段记录索引

| 记录 | 用途 |
|---|---|
| 报告工程的 `evidence/`目录： | |
| workload-build.log | 工作负载与 flash 的构建命令及成功状态（第1.7节摘录） |
| hello-vadd-summary.log | 10 月 9 日运行摘要（第1.7节摘录） |
| rendered/hello-vadd-production-20261025.txt 与 hello-vadd-trace-20261025.txt | 10 月 25 日两次运行的终端输出（第1.7节摘录） |
| 团队仓库的 `team/evidence/stage1-20261009/`： | |
| tool-versions.txt、ubuntu-packages.lock.tsv | 工具版本原始输出与软件包清单 |
| submodules-after-build.lock.txt | 递归依赖完整提交锁 |
| vadd-commit-events.csv、vector-fix-alu-events.csv | 提交记录与三组向量 ALU 输入输出记录，表1-5的来源 |
| wave-extraction.log | 10 月 9 日的波形事件提取记录 |

| 记录 | 用途 |
| --- | --- |
| 团队仓库的 team/evidence/stage1-current-20261025/: | |
| production-run/run.json、trace-run/run.json | 10 月 25 日两次运行的命令、输入摘要、退出状态和日志摘要 |
| wave/wave.json | 采样定义、127 个信号和九条事件 |
| acceptance.json、RESULT.md | 输入等价、24 项工具检查与复现说明 |

复核时先检查运行回执和通过摘要，再核对程序、参考库与 flash 的 SHA-256，最后用反汇编和事件 CSV 定位波形。详细操作入口见阶段一记录、环境部署方法和10 月 25 日复核的复现说明。

## 1.7 操作日志摘录

本节摘录主要的构建和运行输出；版本锁、事件 CSV 和运行回执见表1-10所列位置。

### 1.7.1 10 月 9 日工作负载构建

```
+ /home/riscv/riscv-lab/stage1-build-workload.sh:20: make ARCH=riscv64-xs
MARCH=rv64gcv -j1
# Building hello-vadd [riscv64-xs] with AM_HOME
{/home/riscv/riscv-lab/xs-env/nexus-am}
+ CC main.c
# Building lib-am [riscv64-xs]
+ AS src/nemu/common/mainargs.S
+ AR -> build/am-riscv64-xs.a
# Building lib-klib [riscv64-xs]
# Creating binary image [riscv64-xs]
+ LD -> build/hello-vadd-riscv64-xs.elf
+ OBJCOPY -> build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:21: riscv64-linux-gnu-gcc -c
-march=rv64gc -mabi=lp64d boot-flash.S -o build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:22: riscv64-linux-gnu-objcopy
-O binary -j .text build/boot-flash.o build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:23: riscv64-linux-gnu-objdump
-d build/boot-flash.o


+ /home/riscv/riscv-lab/stage1-build-workload.sh:28: echo WORKLOAD_BUILD_PASS
WORKLOAD_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-workload.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: printf 'exit_code=%s
end=%s\n' 0 2026-10-09T20:55:26+08:00
```

### 1.7.2 10 月 9 日正常运行摘要

```
The image is
/home/riscv/riscv-lab/workloads/hello-vadd/build/hello-vadd-riscv64-xs.bin
The reference model is
/home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/riscv/riscv-lab/workloads/hello-vadd/build/boot-flash.bin
hello xiangshan, 我是 riscv-vdot-team, IPaddress=10.198.93.97
identity_status=placeholder; IPaddress is WSL host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
```

```
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
[PERF ][time=               42965] SimTop.l_soc.l3cacheOpt.slices_0.mshrAlloc:
mshr_lateCore 0: HIT GOOD TRAP at pc = 0x8000029c
Core-0 instrCnt = 16557, cycleCnt = 42962, IPC = 0.385387
Host time spent: 300823ms
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-run-hello-vadd.sh:21: echo
HELLO_VADD_EMU_RUN_PASS
HELLO_VADD_EMU_RUN_PASS
```

### 1.7.3 10 月 25 日生产模型输出

```
emu compiled at Oct 24 2026, 09:22:45
Using simulated 32768B flash
use /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/b
oot-flash.bin as flash bin
Using simulated 128MB RAM
The image is /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/worklo
d/build/hello-vadd-riscv64-xs.bin
The reference model is /home/lizufan/riscv-lab/t17-clean-20261024-0854/support-
01/nemu/reference/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image: /home/lizufan/ri
scv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin

[src/monitor/image_loader.c:250,load_img] Read 12 bytes from file /home/lizufan
/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin to
0x0xf0000000
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu
loads a snapshot)
Host time spent: 56742ms
```

### 1.7.4 10 月 25 日 FST 模型输出

```
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu
loads a snapshot)
Host time spent: 145975ms
```